Backend Unit Tests Report — EduAirControl

María de los Ángeles Olaya García; Jhon Jairo Cortez Vargas

Technology Program in Software Analysis and Development – 3145556, Servicio Nacional de Aprendizaje (SENA)

Software Analysis and Development – 3145556

All instructors of the program

October 13, 2026 · 2025-2026

Abstract

This report describes the state of the **backend unit tests** of EduAirControl: seven Java 17 / Spring Boot 4 services verified on **October 13, 2026** with `./mvnw -B clean verify` on their `develop` branches. The run produced **327 tests with 0 errors and 0 failures** across 45 test classes (≈281 s of accumulated test time).

Of that total, **136 tests in 22 classes are pure unit tests** (JUnit 5 + Mockito + AssertJ, no Spring context and no database): they are the focus of this document. The remaining tests fall into slice tests (`@SpringBootTest` with MockMvc over in-memory H2), one API Gateway integration test with mocked external dependencies, and container-based integration tests against real PostgreSQL and RabbitMQ via Testcontainers. The `infra/scripts/smoke.sh` script **is not a unit test**: it is an end-to-end (E2E) smoke check that requires the full stack to be running and is described in Section 6 so that it is clearly distinguished.

The report ends with six findings, one of which (H5) is now closed, including the absence of coverage measurement (no JaCoCo is configured, even though RNF6 requires ≥ 80 % of lines), the omission of `ms-user-experience` from the continuous-integration matrix, and the risk of false results caused by stale compiled artifacts when the pipeline does not run `clean`.

Keywords: unit tests, JUnit 5, Mockito, Testcontainers, H2, continuous integration, code coverage.

Table of Contents

Abstract 2

1. Introduction 4

2. Definitions: what is and what is not a unit test here 4

3. Run methodology 6

4. Results per service 7

5. Unit test inventory and traceability to the requirements 8

6. What is not unit (and why it matters to distinguish) 12

    6.1 Slice tests over H2 (100 tests) 12

    6.2 Tests with Testcontainers (85 tests) 13

    6.3 The API Gateway with mocked dependencies (6 tests) 13

    6.4 The E2E smoke: infra/scripts/smoke.sh 13

7. Continuous integration 13

8. Findings and recommendations 14

9. Conclusions 15

References 17

Appendix A. Commands of the run 18

Appendix B. Detail by class 18

Backend Unit Tests Report — EduAirControl

1. Introduction

This report documents the actual state of the automated tests of the EduAirControl backend, with emphasis on the **unit tests**. Its purpose is threefold: (a) to answer verifiably which tests exist, how many there are, what they cover and what their status is; (b) to leave a reproducible baseline for the design and implementation phases; and (c) to feed the project's quality strategy (`docs-ea-control/11-quality/`).

The information comes from a fresh run of the test suites of the seven services in the working repository, executed on October 13, 2026, and from the static analysis of each test class (annotations, dependencies and the method under test). The identifiers `RFx.y`, `RNFx` and `CU-xx` are those of the **Software Requirements Specification (Deliverable 02)**, the same ones used by `03_Analisis_Software`, so both pieces share a single nomenclature.

The document is organized as follows: Section 2 (definitions: what is and what is not a unit test in this project), Section 3 (run methodology), Section 4 (results per service), Section 5 (inventory of the unit suites and their traceability to the requirements), Section 6 (what is not unit: slices, Testcontainers and the E2E smoke), Section 7 (continuous integration), Section 8 (findings and recommendations), Section 9 (conclusions) and appendices with the commands and the class-by-class detail.

2. Definitions: what is and what is not a unit test here

To avoid the confusion that motivates this report —«are the unit tests the tests plus smoke.sh?»—, the following table fixes the taxonomy used throughout the document.

Table 1

Taxonomy of the automated backend tests

Type	What it verifies	Isolation	Tool in this project	Where it runs
Pure unit	Domain and application logic in full isolation: aggregates, application services, validators, event builders	Complete: no Spring context, no database, no network; dependencies simulated with Mockito or manual constructors	JUnit 5 + Mockito + AssertJ	`./mvnw verify`
Slice	Controllers and their security configuration with `MockMvc`, over an in-memory H2 database	Medium: minimal Spring context, no real PostgreSQL	`@SpringBootTest` + MockMvc + H2	`./mvnw verify`
Integration with mocks	Full Spring context with external dependencies mocked (Redis, JWKS), no database	Medium-low: real HTTP server, external services mocked	`@SpringBootTest` + WebTestClient + `@MockitoBean`	`./mvnw verify`
Container-based integration	Repositories, RabbitMQ listeners and controllers against **real PostgreSQL** (and real RabbitMQ in monitoring)	Low: real services in Docker	Testcontainers	`./mvnw verify` (requires Docker)
E2E / smoke	Full flows through the API Gateway with the complete stack running	None: real deployed environment	`infra/scripts/smoke.sh` (bash + curl + jq)	Manual, over `docker compose up -d`

With that definition: **the unit tests of this project are the 136 tests of the first row**, not the 327 total, and `smoke.sh` belongs to the last row: it is an end-to-end smoke verification, not a unit test, and it **does not run automatically in any CI pipeline**.

3. Run methodology

Table 2

Conditions of the October 13, 2026 run

Item	Value
Run date and time	October 13, 2026 (UTC-5)
Command executed per service	`./mvnw -B clean verify`
Branch and commit verified	`develop` (after `git pull` of each repository; all up to date)
JDK	OpenJDK 21 (compilation with `--release 17`)
Test engine	Surefire (Maven) inside `verify`
Test database	In-memory H2 (5 services) and in-container PostgreSQL (2 services)
Test containers	Docker Engine 29.7.2 (Testcontainers raises PostgreSQL 15 and, in monitoring, RabbitMQ)
Success criterion	`BUILD SUCCESS` with 0 errors and 0 failures in every service

`clean` was used deliberately: without it, Maven reuses classes compiled in previous executions, and today three services reported **64 spurious errors** due to artifacts of renamed packages; after `clean verify`, all three pass. This fact is recorded as finding H2 (Section 8).

Note on the API Gateway: in the previous run its local working copy contained an uncommitted modification in `RedisStateStoreImpl.java` that broke compilation (finding H5). The edit was finished and committed, the working copy is clean and the suite runs in a normal tree, obtaining 27/27 green.

4. Results per service

Table 3

Results of the October 13, 2026 run (tests / errors / failures / seconds)

Service	Port	Tests	Errors	Fail.	Sec.	Breakdown by type
api-gateway	8080	27	0	0	16.9	21 unit; 6 integration with mocks
ms-security	8081	67	0	0	73.9	28 unit; 39 slice
ms-classroom-management	3002	35	0	0	27.9	10 unit; 25 slice
ms-sensor-management	3004	63	0	0	29.8	41 unit; 22 slice
ms-user-management	3007	22	0	0	24.7	8 unit; 14 slice
ms-environment-monitoring	3003	103	0	0	78.3	28 unit; 75 Testcontainers
ms-user-experience	3006	10	0	0	29.8	10 Testcontainers
Total	—	**327**	**0**	**0**	**281.3**	**136 unit; 100 slice; 6 with mocks; 85 Testcontainers**

Table 4

Summary by test type

Test type	Classes	Tests	% of total	Services
Pure unit	22	136	41.6 %	6
Slice (`@SpringBootTest` + MockMvc/H2)	9	100	30.6 %	4
Integration with mocks (`@SpringBootTest` + WebTestClient)	1	6	1.8 %	1
Testcontainers (real PostgreSQL/RabbitMQ)	13	85	26.0 %	2

Test type	Classes	Tests	% of total	Services
Total	**45**	**327**	**100 %**	**7**

The distribution resembles the base of the testing pyramid: a wide floor of lightweight tests (unit + slice = 72.2 %) and a small top of heavy integration. What is missing is the coverage measurement that would tell what percentage of the code that floor executes (finding H1).

5. Unit test inventory and traceability to the requirements

The 22 pure unit classes are detailed below with the behaviour they verify and their correspondence with the requirements and use cases of Deliverable 02 (numbering from `03_Analisis_Software`).

Table 5

Pure unit suites, verified behaviour and covered requirements

Service	Class	T.	What it verifies	Requirements (Deliverable 02)
api-gateway	`JwtValidatorTest`	6	JWT validation at the edge: token signed with another key, with a tampered `alg` or empty is rejected; public paths exempt	RF1.8, RF2.3 (RNF4)
api-gateway	`RateLimitFilterTest`	6	Rate limiting at the edge: requests under the threshold allowed, blocked once exceeded, keyed by IP for anonymous and by `userId` for authenticated callers, exempt paths, and fail-open when Redis is unavailable	RNF1, RNF4
api-gateway	`PublicPathsTest`	4	Public/private boundary: `/health` and the authentication entry points, OAuth2 and preflight paths exempt; `logout` and business routes require a token	RF1.8

Service	Class	T.	What it verifies	Requirements (Deliverable 02)
api-gateway	`RemoteJwksProviderTest`	3	JWKS key resolution: unknown `kid` rejected, public key fetched and keys cached between calls	RF1.9, RNF4
api-gateway	`CorrelationIdFilterTest`	2	Request correlation: generates `X-Correlation-Id` when the client does not send one and preserves an incoming value	RNF5
ms-security	`AuthServiceTest`	11	Registration and sign-in: duplicate email, unknown institution and weak password rejected; user created with role and tokens; the fifth failed login locks the account (423); `refresh` rotates and revokes the previous token; `refresh` on a revoked token rejected; `logout` revokes one device or all of them	RF1.1, RF1.3, RF1.9
ms-security	`JwtServiceTest`	7	RS256 issuance: correct claims, `kid` consistent with the JWKS, expiry matching the TTL, tampered and expired tokens rejected, RSA modulus and exponent of the JWKS	RF1.1, RF1.9
ms-security	`RedisTokenBlacklistTest`	3	Revocation list: blank or already-expired `jti` skipped, valid `jti` persisted with its remaining TTL	RF1.9
ms-security	`InstitutionServiceTest`	5	Institution creation: uppercase code, duplicate rejected, blank name rejected, 404 for unknown ID, partial update	RF2.1
ms-security	`SuperAdminBootstrapTest`	2	`SUPER_ADMIN` bootstrap: creates the user with its role when missing and does not duplicate it when it already exists	RF1.8

Service	Class	T.	What it verifies	Requirements (Deliverable 02)
ms-classroom-management	`CampusServiceTest`	5	Campuses: duplicate code rejected, normalized code, soft delete, 404 for deleted campus	RF2.2
ms-classroom-management	`EducationalEnvironmentCreatedEventTest`	5	Contract of the `educational.environment.created` event: real aggregate data, `aggregateId`, routing key, event envelope and unique `eventId`	RF8.1, RF3.1
ms-user-management	`UserProfileServiceTest`	8	Profiles: duplicate `userId` rejected, trimmed text, required full name, update of provided fields, soft delete	RF7.1, RF2.4
ms-sensor-management	`SensorServiceTest`	8	Sensors: unique serial, trimmed serial, delete blocked with installation history (409), unknown model and status rejected	RF8.2
ms-sensor-management	`SensorInstallationServiceTest`	12	Installations: overlap, unknown sensor and invalid close rejected; open installation persisted; close sets `removedAt`; published (or not) of installed/removed events; **physical delete** that removes the row, publishes its event and also applies to an installation that is still active	RF8.2

Service	Class	T.	What it verifies	Requirements (Deliverable 02)
ms-sensor-management	`SensorCatalogServiceTest`	9	Catalog: models, statuses and units; delete blocked while the element is in use	RF8.2
ms-sensor-management	`VariableServiceTest`	5	Variables: duplicate code, trimmed text, delete blocked when referenced by sensors, required measurement unit	RF8.2, RF3.2
ms-sensor-management	`SensorInstallationEventTest`	7	Contract of the installation events: payload, timestamps, routing key, envelope readable by the consumer and unique `eventId`; the deleted event is distinguishable from the closed one	RF8.2, RF9.1
ms-environment-monitoring	`EnvironmentalAnalysisTest`	10	State machine of the environmental analysis: `PENDING` → `RUNNING` → `COMPLETED/FAILED`, non-empty windows, results attached only while running	RF3.4
ms-environment-monitoring	`PeriodResolverTest`	12	Resolution of day/week/month/year periods, leap year, December-to-January rollover, valid window, null period rejected	RF3.4, INV-007

Service	Class	T.	What it verifies	Requirements (Deliverable 02)
ms-environment-monitoring	`EnvironmentAlertTest`	3	Alerts: new alert is active, acknowledge marks it inactive and cannot be repeated	RF6.2
ms-environment-monitoring	`EnvironmentalDataAnalyzedTest`	3	Analysis result event: payload, normalization of null results and defensive copy	RF3.4

Three traceability observations: (a) the event-contract classes (`*EventTest`) protect the asynchronous boundary between services, where coupling is most fragile; (b) the security requirements (RF1.1, RF1.8, RF1.9) now have **complete** unit coverage — `AuthServiceTest` covers account lockout and refresh-token rotation, which in the previous run were only tested in the slices; (c) `ms-user-experience` has no pure unit test at all — its 10 tests are all Testcontainers — which is why the first row of Table 4 spans six services rather than seven.

6. What is not unit (and why it matters to distinguish)

6.1 Slice tests over H2 (100 tests)

Nine `@SpringBootTest` classes with MockMvc start the full Spring context of the service over in-memory H2 (Liquibase disabled, schema created by Hibernate). They cover the controllers: `AuthControllerTest` (20), `CampusControllerTest` (17), `SensorControllerTest` (16), `UserProfileControllerTest` (14), `AdminControllerTest` (10), `EducationalEnvironmentControllerTest` (8), `SensorFlowControllerTest` (6),

`PasswordFlowTest` (5) and `OAuth2EndpointTest` (4). They are fast, but they depend on the application context: they must not be confused with unit tests.

6.2 Tests with Testcontainers (85 tests)

`ms-environment-monitoring` and `ms-user-experience` validate repositories, RabbitMQ listeners and controllers against **real PostgreSQL 15** (and real RabbitMQ in monitoring) through Testcontainers. They include `SchemaVerificationTest` (9), which verifies that the generated schema matches the expected model, and `ThresholdServiceTest` (7), which exercises threshold evaluation against real data. They are the slowest (78.3 s and 29.8 s) and **require Docker**: without it, those two services cannot verify their full suite.

6.3 The API Gateway with mocked dependencies (6 tests)

`GatewayIntegrationTest` is the only class that boots a full Spring context **without a database**: it raises the gateway's HTTP server and runs the real filters (JWT, correlation, rate limit) against Redis and a JWKS provider stubbed with `@MockitoBean`. It is neither an H2 slice nor a container test, which is why it is counted separately in Table 4.

6.4 The E2E smoke: `infra/scripts/smoke.sh`

The script is neither a unit test nor does it run in CI. It is an **end-to-end** smoke verification against the running stack (`docker compose up -d`) that requires `jq` and the SUPER_ADMIN seeded at boot. Sequence: `GET /health` of the gateway → SUPER_ADMIN login → creation of a temporary institution → registration of a user with that institution's code → login → `logout` (revocation) → retry with the revoked token, which **must** return `401 TOKEN_REVOKED`. It covers the critical identity flow (CU-02, RF1.1, RF1.9) end to end, something no unit test can do.

7. Continuous integration

The active pipeline lives in `infra/.github/workflows/build-services.yml`. For each `EduAirControl/ea-control-*` repository it checks out the code, sets up JDK 17 (Temurin) and runs `./mvnw -verify`; if the event is a push to `main`, it builds and publishes the image to GHCR.

Table 6

CI matrix coverage versus existing services

Service	In the CI matrix?	Runs with `clean`	Comment
api-gateway	Yes	No	Risk H2
ms-security	Yes	No	Risk H2
ms-classroom-management	Yes	No	Risk H2
ms-sensor-management	Yes	No	Risk H2
ms-user-management	Yes	No	Risk H2
ms-environment-monitoring	Yes	No	Requires Docker for Testcontainers (the runner provides it)
ms-user-experience	**No**	—	**Missing from the matrix** (finding H3)

`smoke.sh` does not appear in any pipeline: it is run manually (finding H4).

8. Findings and recommendations

Table 7

Findings of the October 13, 2026 review

#	Finding	Evidence	Impact	Recommendation
H 1	No code-coverage measurement	Zero JaCoCo configurations in the seven `pom.xml`	RNF6 (≥ 80 % of lines, ≥ 90 % of business logic) cannot be verified	Add the JaCoCo plugin with a failing gate in the pipeline

#	Finding	Evidence	Impact	Recommendation
H 2	CI runs `verify` without `clean`	In the previous run, 3 services produced 64 spurious errors from stale classes; with `clean` they pass	False reds or false greens depending on the runner	Switch the matrix to `./mvnw -B clean verify`
H 3	`ms-user-experience` is not in the CI matrix	The matrix lists 6 services; the seventh exists and has 10 tests	Its tests do not run on each push/PR	Add the service to the matrix
H 4	The E2E smoke is not automated	`smoke.sh` is only invoked manually (infra README)	Verification of the critical flow depends on a person	Run it in the staging pipeline or as a manual job (`workflow_dispatch`)
~~ H 5 ~~	~~The `api-gateway` working copy does not compile~~ **CLOSED**	The edit in `ratelimit/RedisStateStoreImpl.java` was finished and committed; the working copy is clean and the suite runs without an isolated worktree (27/27)	—	None
H 6	`MICROSERVICIOS.md` is outdated	It states «139 tests» and marks `ms-environment-monitoring` and `ms-user-experience` as pending	Misleading information for anyone joining the project	Update the count to 327 and the service statuses

9. Conclusions

1. The backend unit tests are **green**: 136 pure unit tests in 22 classes, spread across six of the seven services, with 0 errors and 0 failures in the October 13, 2026 run.

2. The backend as a whole has **327 automated tests** (unit, slice, with mocks and container-based integration), a figure that triples the one declared in the internal documentation and must be updated (finding H6).
3. The `smoke.sh` **is not a unit test**: it is the E2E verification of the identity flow and should be treated as a distinct, complementary and until now manual layer.
4. The main gap is not the number of tests but **measurement**: without JaCoCo there is no coverage percentage and requirement RNF6 is not verifiable (finding H1).
5. Two one-line fixes have immediate return: `clean` in the pipeline (H2) and `ms-user-experience` in the CI matrix (H3).

References

Docker. (n.d.). *Docker documentation*. https://docs.docker.com

EduAirControl. (2026). *Software requirements specification of EduAirControl* (Deliverable 02) [Internal project document].

EduAirControl. (2026). *Backend unit tests report* (Deliverable 07) [Internal project document].

JUnit. (n.d.). *JUnit 5 documentation*. https://junit.org/junit5/docs/current/user-guide/

Mockito. (n.d.). *Mockito*. https://mockito.org

Maven. (n.d.). *Surefire plugin*. https://maven.apache.org/surefire/

OWASP. (n.d.). *OWASP Top Ten*. https://owasp.org/www-project-top-ten/

Sonar. (n.d.). *JaCoCo*. https://www.jacoco.org/jacoco/

Testcontainers. (n.d.). *Testcontainers*. https://testcontainers.com

Appendix A. Commands of the run

```
# Verify a service (H2, no Docker)
cd ms-security && ./mvnw -B clean verify

# Verify the two services with Testcontainers (requires Docker)
cd ms-environment-monitoring && ./mvnw -B clean verify
cd ms-user-experience && ./mvnw -B clean verify

# Check the API Gateway (clean working copy, no isolated worktree)
cd api-gateway && ./mvnw -B clean verify

# E2E smoke (requires the full stack and jq)
cd infra && docker compose up -d && ./scripts/wait-for-services.sh
&& ./scripts/smoke.sh
```

Appendix B. Detail by class

Table 8

The 45 test classes with their tests and type

Service	Class	T.	Type
api-gateway	JwtValidatorTest	6	Pure unit
api-gateway	GatewayIntegrationTest	6	Integration with mocks
api-gateway	RateLimitFilterTest	6	Pure unit
api-gateway	PublicPathsTest	4	Pure unit
api-gateway	RemoteJwksProviderTest	3	Pure unit
api-gateway	CorrelationIdFilterTest	2	Pure unit
ms-security	AuthControllerTest	20	Slice
ms-security	AuthServiceTest	11	Pure unit
ms-security	AdminControllerTest	10	Slice
ms-security	JwtServiceTest	7	Pure unit
ms-security	PasswordFlowTest	5	Slice

Service	Class	T.	Type
ms-security	InstitutionServiceTest	5	Pure unit
ms-security	OAuth2EndpointTest	4	Slice
ms-security	RedisTokenBlacklistTest	3	Pure unit
ms-security	SuperAdminBootstrapTest	2	Pure unit
ms-classroom-management	CampusControllerTest	17	Slice
ms-classroom-management	EducationalEnvironmentControllerTest	8	Slice
ms-classroom-management	CampusServiceTest	5	Pure unit
ms-classroom-management	EducationalEnvironmentCreatedEventTest	5	Pure unit
ms-sensor-management	SensorControllerTest	16	Slice
ms-sensor-management	SensorInstallationServiceTest	12	Pure unit
ms-sensor-management	SensorCatalogServiceTest	9	Pure unit
ms-sensor-management	SensorServiceTest	8	Pure unit
ms-sensor-management	SensorInstallationEventTest	7	Pure unit
ms-sensor-management	SensorFlowControllerTest	6	Slice
ms-sensor-management	VariableServiceTest	5	Pure unit
ms-user-management	UserProfileControllerTest	14	Slice
ms-user-management	UserProfileServiceTest	8	Pure unit
ms-environment-monitoring	MonitoringControllerTest	17	Testcontainers
ms-environment-monitoring	AnalysisControllerTest	12	Testcontainers
ms-environment-monitoring	PeriodResolverTest	12	Pure unit

Service	Class	T.	Type
ms-environment-monitoring	EnvironmentalAnalysisTest	10	Pure unit
ms-environment-monitoring	HistoricalBackfillServiceTest	10	Testcontainers
ms-environment-monitoring	SchemaVerificationTest	9	Testcontainers
ms-environment-monitoring	ThresholdServiceTest	7	Testcontainers
ms-environment-monitoring	SensorInstallationListenerTest	6	Testcontainers
ms-environment-monitoring	ClassroomEnvironmentListenerTest	5	Testcontainers
ms-environment-monitoring	AlertServiceTest	4	Testcontainers
ms-environment-monitoring	MeasurementAlertingServiceTest	4	Testcontainers
ms-environment-monitoring	EnvironmentAlertTest	3	Pure unit
ms-environment-monitoring	EnvironmentalDataAnalyzedTest	3	Pure unit
ms-environment-monitoring	MsEnvironmentMonitoringApplicationTests	1	Testcontainers
ms-user-experience	UserExperienceControllerTest	7	Testcontainers
ms-user-experience	UxBackfillServiceTest	2	Testcontainers
ms-user-experience	MsUserExperienceApplicationTests	1	Testcontainers